SMU Classification: Restricted

Web Application Development II

Week 6: Vue.js Modularizatoin with Components

Agenda

Introduction to Components

Defining a Component

Registration and using a component

Dynamic properties

Passing data to component

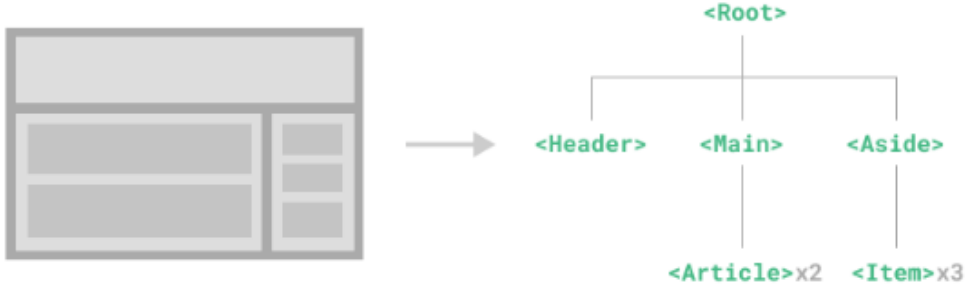
Persisting data with Local Storage



Components

Components allow us to split the UI into independent and reusable pieces, and think about each piece in isolation. It's common for an app to be organized into a tree of nested components.

This is very similar to how we nest native HTML elements, but Vue implements its own component model that allows us to encapsulate custom content and logic in each component.



Naming Convention

It is recommended to use PascalCase for ComponentNames **<PascalCase />** to differentiate from native HTML elements.

PascalCase names are valid JavaScript identifiers (kebab-case names are not). This makes it easier to import and register components in JavaScript. It also helps IDEs with auto-completion.

On the other hand, a component registered as MyComponent can be referenced inside a Vue template via both <MyComponent> and <my-component>.

Defining a Component

Using Composition API, we can define each Vue component in a dedicated file using the .vue extension - known as a Single-File Component (SFC for short):

Example: *components/ButtonCounter.vue*

ChildComponent.vue

```
<script setup>
import { ref } from 'vue'
const count = ref(0)
</script>

<template>
  <button @click="count++">You clicked me {{ count }} times.</button>
</template>
```

Using a Component

To use a child component, we need to import it in the parent component.

Components can be reused as many times as you need

Example: *views/Example1View.vue*

ParentComponent.vue

```
<script setup>
// naming convention: PascalCase
import ButtonCounter from '../components/ButtonCounter.vue'
</script>

<template>
<!– can use kebab case or PascalCase in template -->
   <!-- <buttoncounter /> --> <!-- this does not work -->
   <ButtonCounter /> <!– PascalCase -->
   <button-counter /> <!– kebab-case -->
</template>
```

src/components/ vs src/views/

Now that you are learning about components, it is now a good time to make use of **views/** folder in our Vue SPA setup.

Notice in our example folder in this week that we now place example files in **views/** folder.

Rule of Thumb:
If it is a page that has a URL, put it in **views/**
If it is a piece of UI you can reuse, put it in **components/**

src/components/ – reusable building blocks
It stores reusable UI components that can be used in many places across the app, e.g., NavBar.vue, Header.vue, Footer.vue, PostCard.vue, etc.

src/views/ – route-level pages (screens)
It stores page/view components that represent full screens in your SPA, usually mapped via Vue Router routes (*router/index.js*)
E.g., HomeView.vue, PostsView.vue, LoginView.vue, etc.

Template

template serves as the markup for the Vue component

Intuitively, the template is used to replace the component-element.

<ButtonCounter />

Example: *components/ButtonCounter.vue*

```
<template>
  <p>If you like my page, click </p>
  <button @click="count++">Like</button>
  <p>People liked me {{ count }} times.</p>
  <p v-if="count>2"> I am POPU-LAR.</p>
</template>
```

Passing Props (1)

Props are custom attributes that are exposed to accept data from the parent component.

For example, if you are creating a custom card, the card will need the content (text data) from the parent component, so that the cards can have the same visual layout but with different content.

A component can have as many props and value of various data type can be passed to the props.

To use Props, you need to register on the component using **defineProps**

Example: *components/Product.vue*

```
<script setup>
  const props = defineProps(['name', 'price])
  console.log(props.name)
</script>

<template>
   <h3>Product: {{ name }}</h3>
</template>
```

props are (custom) HTML attributes

Passing Props (2)

Once a prop is registered, you can pass data to it as a custom attribute from parent component

Example: *views/Example2View.vue*

```
<script setup>
  import Product from
          '../components/Product.vue'
</script>

<template>
   <product name="iPhone' price='100' />
</template>
```

props are (custom) HTML attributes

Passing Props (3)

In addition to declaring props using an array of strings, we can also use **the object syntax**

The key is the name of the prop

The value is **the expected data type**, e.g., Object, String, Number, Boolean

Naming convention when declaring a prop: **camelCase**

Example

ChildComponent.vue

```
<script setup>
   const props = defineProps({
                 greetingMessage: String
               })
   console.log(props.greetingMessage)
</script>


<template>
    <div v-html='greetingMessage'></div>
</template>
```

accessing a prop

Passing Props (4)

But to align with HTML attributes, naming convention when using a prop: kebab-case

Example

ParentComponent.vue

```
<script setup>
   import ChildComponent from '../components/ChildComponent.vue'
</script>


<template>
    <ChildComponent greeting-message="hello" />
</template>
```

Accessing Props

Example: *components/Product.vue*

In JavaScript context, you can access the props via props.<propname>

In HTML context (template), you can access the props just like accessing a JavaScript variable

```
<script setup>
  const props = defineProps(['name', 'price])
  console.log(props.name)
</script>

<template>
   <h3>Product: {{ name }} </h3>
</template>
```

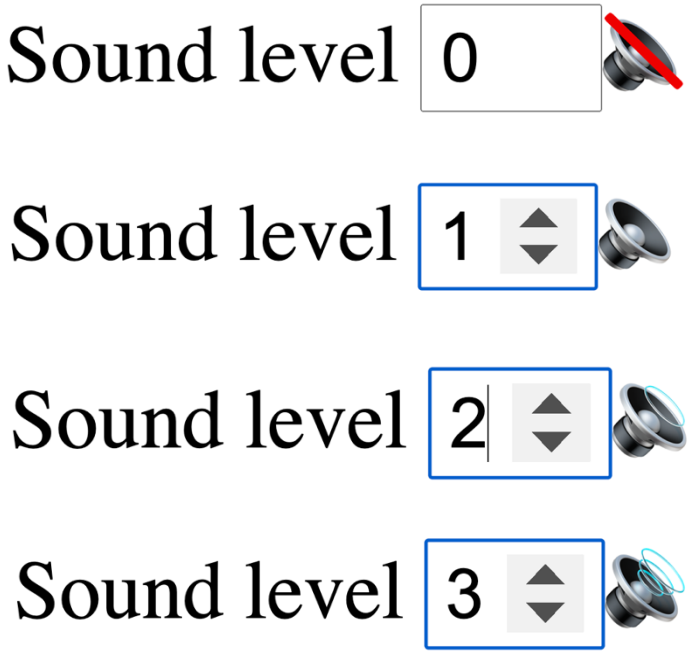
accessing a prop

Exercise 1

In this exercise, we would like to define a component for displaying “Sound Level”.

Given Ex1View.vue and SoundIcon.vue, complete the TODO such that when the user enters a value between 0 and 3, a corresponding sound-level icon appears on the right.

Be mindful how values are passed around.



Dynamic Properties (1)

In a typical app, you may need to pass an array of data from your parent component.

In this case, use **v-for** and **v-bind**

Note that when using v-for, it is recommended to bind the **key** attribute with a unique value, so that Vue can correctly track each node based on the unique value. Refer to this for detail

Example: *views/Example3View.vue*

```
// use component
const products = ref([
      { id: 1, name: 'iPhone',
          desc: '<p>This is the best phone in the world!</p>' },
      { id: 2, name: 'Samsung',
          desc: '<p>No, mine is the best!</p>' },
      { id: 3, name: 'Oppo',
          desc: '<p>Mine is cheaper and yet as good!</p>' }
   ])

====
<Product
      v-for="product in products"
      v-bind:key="product.id"
      v-bind:prod="product"
/>
```

Dynamic Properties (2)

Notice in previous slide how **v-bind:prod="product"** is used to pass dynamic prop values.

This is useful when you don't know the exact content you are going to render ahead of time, i.e., dynamic data

Dynamic data may be obtained from user or Asynchronous requests

Example: *components/Product2.vue*

```
// define component
<script setup>
  const props = defineProps({
                prod: Object
              })
</script>

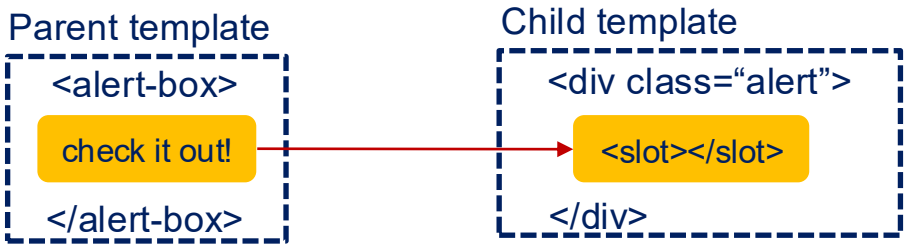
<template>
  <h3> {{ prod.id }} : {{ prod.name }}  </h3>
</template>
```

Passing Content via Slot

Just like with HTML elements, it's often useful to be able to pass content to a component.

This can be done with <slot> element.

Use named-slots when there are multiple slots.



Example: *Example4View.vue, AlertBox.vue, Book.vue*

```
// define (AlertBox.vue)
<div class="alert">
    <slot></slot>
</div>

// use (Example4.vue)
<alert-box>
    check it out!
</alert-box>
```

Named Slots:

```
<div>
    <slot name='slotA'></slot>
    <slot name='slotB'></slot>
</div>
```

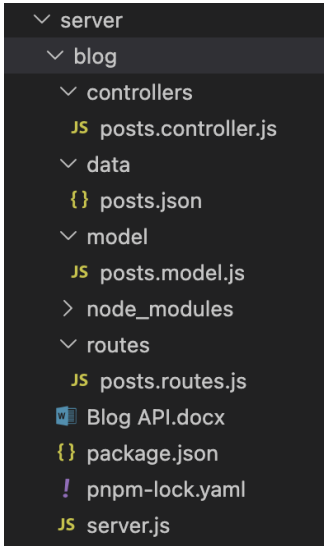
(Week5) Running example: A Blog Service (1)

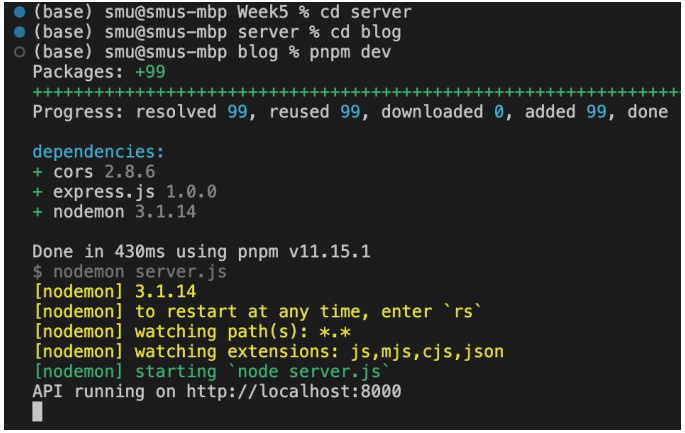
Set up a blog backend service using **Node.js + Express.js**

These technologies are covered in WAD1.

1. Download **server.zip** and unzip into a folder, say *server.* It contains a folder 'blog' which is a blog backend app.
2. In the terminal, run the following commands to launch this backend service

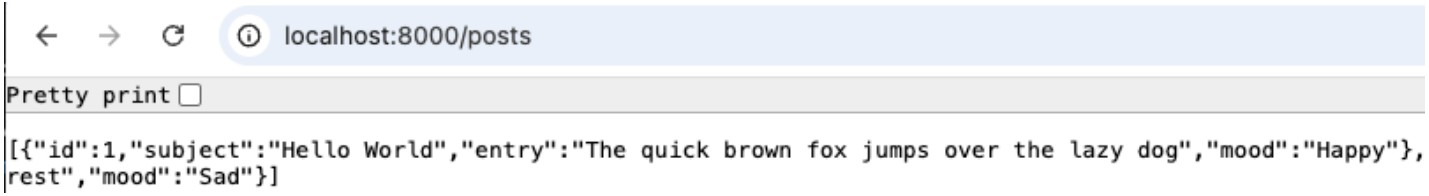
```
cd server/blog
pnpm install
pnpm dev
```





(Week5) Running example: A Blog Service (2)

3. In the browser, run the URL: http://localhost:8000/posts
You should observe the JSON data returned by the blog service.



4. This blog service provides a few APIs. Some of them are:

retrieve all posts: GET /posts
retrieve post with a given id: GET /posts/:id
create a new post: POST /posts
update an existing post with a given id: PUT /posts/:id
delete an existing post with a given id: DELETE /posts/:id

Example *retrieve post with a given id* =>



SMU Classification: Restric

Exercise 2

Given Ex2View.vue and BlogPost.vue, complete the TODO so that the posts are displayed using ‘BlogPost’ components.

Hint:

three images are provided under the folder /src/assets/ for different moods.

Use computed property to bind the image source







Exercise 3

After completing Ex2View.vue in Exercise 2, add a ‘delete’ button for each post such that the post can be deleted using the deletePost service given in “blog” application (i.e.,
send Asynchronous DELETE request to
http://localhost:8000/posts/:id

with the blog ‘subject’ parameter)

Hint:

- Use **slot** to add the button
- Use Axios delete method
    - axios.delete(url, { data: { subject: subject } })



Emitting Events (1)

Sometimes the nodes (childs) in the component may need to communicate to the parent, such as ‘emit an event’

Suppose we have a button in a component. On clicking the button, it wants to enlarge the text. So the component needs to declare that it will emit events using **defineEmits()**

Naming convention for event name: same as ‘props’

Example: *Product3.vue*

```
const emit = defineEmits(['enlarge', 'shrink'])
```

Emitting Events (2)

We can emit an event either inline or inside a function

For inline emitting, use **$emit** to emit a custom event

Example: *Product3.vue*

```
<div class='product'>
  <h3>{{ prod.name }}</h3>
  <button v-on:click="$emit('enlarge')">
            Enlarge text
  </button>
  <div v-html='prod.desc'></div>
</div>
```

Emitting Events (3)

We can emit an event either inline or inside a function

For emitting an event inside a function, use the returned **emit instance** from defineEmits()

Example: *Product3.vue*

```
<script>
const emit = defineEmits(['enlarge', 'shrink'])

function doShrink() {
   emit('shrink')
}
</script>

<template>
      <button v-on:click="doShrink">
         Shrink text
      </button>
</template>
```

Listening to Events (1)

Similar to listening to HTML events, the parent component can listen to the (custom) events emitted by the child component(s) it uses.

Example: *Example5View.vue*

```
<template>
   <div :style="{ 'font-size': fontSize + 'em' }">
      <Product v-for="product in productData“
                  :key="product.id” :prod="product"
                  @enlarge="fontSize += 0.1"
                  @shrink="fontSize -= 0.1"
      />
   </div>
</template>
```

Event Arguments (1)

It is sometimes useful or necessary to emit a specific value with an event.

For example, we may want
the <Product> component to be in charge of how much to enlarge the text by.

In those cases, we can pass an extra argument to provide this value:

Example: *Product4.vue*

```
<div class='product'>
  <h3>{{ prod.name }}</h3>

  <button v-on:click =
            "$emit('enlarge', 0.1)">
     Enlarge text </button>

  <div v-html='prod.desc'></div>
</div>
```

Event Arguments (2)

It is sometimes useful or necessary to emit a specific value with an event.

For example, we may want
the <Product> component to be in charge of how much to enlarge the text by.

In those cases, we can pass an extra argument to provide this value:

Example: *Product4.vue*

```
// emit event from a function
function doShrink() {
   // notice the use of object as emitted data
   emit('shrink', { amount: 0.1 })
}
```

Listening to Events (2)

Similar to listening to HTML events, the parent component can listen to the (custom) events emitted by the child component(s) it uses.

It can also retrieve the emitted value inside the handler function:

Example: *Example6View.vue*

```
function onEnlarge(enlargeAmount) {
    fontSize.value += enlargeAmount
}
function onShrink(shrink) {
    fontSize.value -= shrink.amount
}
====
<div :style="{ 'font-size': fontSize + 'em' }">
    <Product v-for="product in products"
     :key="product.id" :prod="product"
    @enlarge="onEnlarge“ @shrink="onShrink"
    />
</div>
```

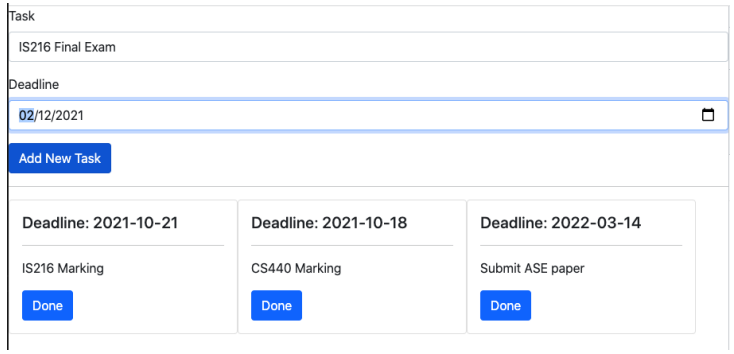
Exercise 4

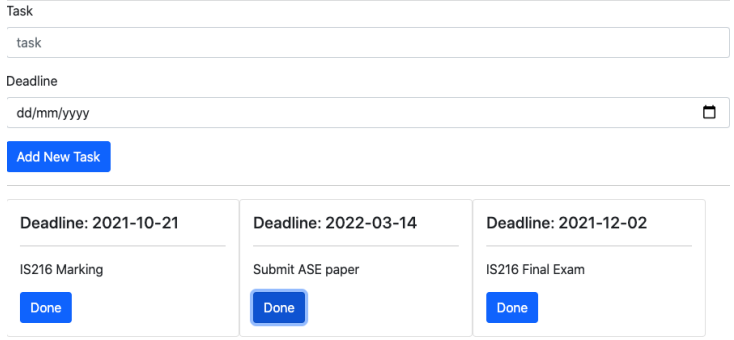
Add/modify code in Ex4View.vue and TaskTracker.vue such that when the user enters a new task and deadline and click “Add New Task”, it displays the newly-entered task in the web page.

When the user clicks “Done”, the task is removed.

You must use the given TaskTracker component to implement the function described above.

Hint: Use **$emit** to generate events from component; use Bootstrap Card.





Global Registration

In our previous examples, components are imported only locally, i.e., a parent component imports a child component to be used in its local template.

We can also make components available globally using **.component()** method

Globally registered components can be used in the template of any component within the application

Example: *main.js, App.vue, HomeView.vue*

```
import ComponentA from ‘./components/ComponentA’
import ComponentB from ‘./components/ComponentB’
import ComponentC from ‘./components/ComponentC’

app.component(‘ComponentA’, ComponentA)
app.component(‘ComponentB’, ComponentB)
app.component(‘ComponentC’, ComponentC)
```

=====

```
<!-- this will work in any component inside the app -->
<ComponentA />
<ComponentB />
<ComponentC />
```

Persisting Data with Local Storage (1)

Sometimes, a client app like Vue.js application needs to save and load app state locally (in the client's machine).

For example, to persist data across page refresh, browser restart, remember UI preferences such as dark mode, language, etc.

Note: avoid storing sensitive data such as passwords, API keys in localStorage as any JS on the page can read it

Example: *Example7View.vue*

Methods:

localStorage.getItem(key) returns string | null

localStorage.setItem(key, value) stores strings only

localStorage.removeItem(key) removes the storage under the given key

onMounted() is a good place to load initial values

Persisting Data with Local Storage (2)

Modern browser provide a local storage mechanism using key-value pairs (both stored as strings).

Scope: per origin (same protocol + domain + port), e.g., http://localhost:5173

Example: *Example7View.vue*

Methods:

localStorage.getItem(key) returns string | null

localStorage.setItem(key, value) stores strings only

onMounted() is a good place to load initial values

Persisting Data with Local Storage (3)

localStorage only supports “string”. If you wish to save an object or an array, use JSON.stringify and JSON.parse to serialize and deserialize object/array

Serialization is the process of converting object/array into a standardized, portable format (like text) so it can be stored or transmitted. Deserialization is the exact reverse process.

Example

```
const obj = { name: “Jack” }
// save data
localStorage.setItem( key, JSON.stringify(obj) )

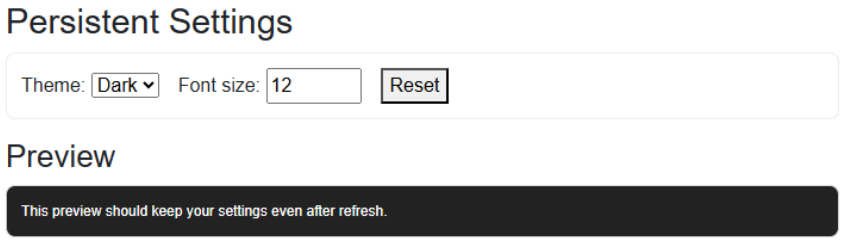
// load data
const data = JSON.parse( localStorage.getItem(key) )
```

Exercise 5

Add/modify code in Ex5View.vue and SettingPanel.vue to persist an object { theme, fontSize } in localStorage using event handlers.

Requirements:

- Save settings whenever the user changes:
  - Theme dropdown (@change)
  - Font size input (@input or @change)
- Load settings on startup using onMounted
- Reset clears both UI state and localStorage



Take Away Message

Define reusable templates as components.

Use props, slots to pass data to components.

Use $emit and custom events, with argument, to pass data from component back to its parents.

Use localStorage mechanism provided by Modern browsers to persist (non-sensitive) data such as user preferences